Amazon CloudWatch — Console Dashboard Wise Important Interview Questions & Answers

CloudWatch Console Structure

```
CloudWatch
│
├── Home / Overview
│
├── Dashboards
│   ├── Dashboards
│   └── Automatic Dashboards
│
├── Metrics
│   ├── All Metrics
│   ├── AWS Namespaces
│   ├── EC2
│   ├── EBS
│   ├── RDS
│   ├── ELB
│   ├── S3
│   ├── Custom Metrics
│   └── Metric Math
│
├── Alarms
│   ├── All Alarms
│   ├── In Alarm
│   ├── OK
│   └── Insufficient Data
│
├── Logs
│   ├── Log Groups
│   │   └── Log Group
│   │       ├── Log Streams
│   │       ├── Metric Filters
│   │       ├── Subscription Filters
│   │       └── Retention
│   ├── Live Tail
│   ├── Logs Insights
│   └── Logs Anomaly Detection
│
├── Application Signals
│   ├── Services
│   ├── Application Map
│   ├── Service Level Objectives
│   └── Service Details
│
├── Synthetics
│   ├── Canaries
│   └── Canary Runs
│
├── Network Monitoring
│   ├── Network Synthetic Monitors
│   └── Network Monitoring
│
├── Insights
│   ├── Container Insights
│   ├── Lambda Insights
│   └── Application Insights
│
├── Events / EventBridge
│
└── Settings
```

---

1. What is Amazon CloudWatch?
Answer:
Amazon CloudWatch is an AWS monitoring and observability service.
It collects metrics, logs, events, and other monitoring data from AWS resources and applications.
We use it to monitor resource health, troubleshoot problems, and create alerts.

For example:
Suppose our application is running on EC2.
CloudWatch can monitor CPU usage and application logs from that EC2 instance.
If CPU becomes too high, CloudWatch can send an alarm notification.
Example:
EC2
↓
CloudWatch
↓
Metrics + Logs
↓
Alarm
↓
Notification
Simple ga gurthupettuko:
CloudWatch = Monitor + Logs + Metrics + Alerts

---

2. What is the CloudWatch Dashboard?
Answer:
A CloudWatch Dashboard is a visual page where we can see important metrics and alarms in one place.
It helps us monitor multiple AWS resources without opening every service separately.
We can create customized dashboards according to our application requirements.
For example:
Suppose our application has EC2, ALB, and RDS.
We can create one CloudWatch dashboard showing EC2 CPU, ALB requests, and RDS CPU together.
This makes application monitoring easier.
Example:
CloudWatch Dashboard
↓

```
┌──────────────┐
│ EC2 CPU      │
│ ALB Requests │
│ RDS CPU      │
│ Alarms       │
└──────────────┘
```

Simple ga gurthupettuko:
Dashboard = All important monitoring information in one screen
CloudWatch dashboards can also combine metrics and alarms across resources, Regions, and accounts when configured for cross-account observability. **A** AWS Documentation

---

3. What are Automatic Dashboards in CloudWatch?
Answer:
Automatic dashboards are dashboards that CloudWatch creates for certain AWS services or monitoring data.
They provide predefined views of important metrics without requiring us to manually build every widget.
They are useful for quickly checking the health of AWS resources.
For example:
Suppose our account has CloudWatch Logs producing supported metrics.
CloudWatch can provide an automatic dashboard showing related monitoring information.
We can use it for quick troubleshooting.
Example:
AWS Resource
↓
CloudWatch
↓
Automatic Dashboard
↓
Metrics / Health
Simple ga gurthupettuko:
Automatic Dashboard = Prebuilt monitoring view

---

4. What are CloudWatch Metrics?
Answer:
A metric is a numerical measurement collected over time.
CloudWatch uses metrics to monitor the performance and health of AWS resources and applications.
Examples include CPU utilization, network traffic, request count, latency, and error count.
For example:

Suppose an EC2 server is running our application.
CloudWatch can collect the EC2 CPUUtilization metric every monitoring period.
We can use that metric to understand whether the server is overloaded.
Example:
EC2
↓
CPUUtilization
↓
CloudWatch Metric
↓
Graph
Simple ga gurthupettuko:
Metric = Numerical measurement

---

5. What is a Metric Namespace?
Answer:
A namespace is a container used to organize related CloudWatch metrics.
AWS services publish their metrics under service-specific namespaces.
For example, EC2 metrics are available under the AWS/EC2 namespace.
For example:
Suppose we want to monitor an EC2 instance.
In CloudWatch Metrics, we select the AWS/EC2 namespace.
Then we can select the required EC2 metrics.
Example:
CloudWatch Metrics
↓
AWS/EC2
↓
CPUUtilization
NetworkIn
NetworkOut
Simple ga gurthupettuko:
Namespace = Group/container for metrics


CloudWatch lets you browse metrics by namespace and dimension from **Metrics → All metrics**. **A** AWS Documentation

---

6. What are Dimensions in CloudWatch Metrics?
Answer:
Dimensions are additional information used to identify or separate metrics.
They help CloudWatch distinguish metrics for different resources or categories.
For example, an EC2 metric can be associated with a particular InstanceId.
For example:
Suppose we have three EC2 instances.
CloudWatch can use InstanceId as a dimension to distinguish each instance's CPU metric.
This lets us monitor each instance separately.
Example:
EC2 CPU Metric
↓
InstanceId
├— i-111
├— i-222
└— i-333
Simple ga gurthupettuko:
Dimension = Identifies which resource the metric belongs to

---

7. What are important AWS CloudWatch metrics?
Answer:
Different AWS services publish different metrics to CloudWatch.
For EC2, common metrics include CPUUtilization, NetworkIn, NetworkOut, and status-related metrics.
For services such as EBS, RDS, and ELB, CloudWatch provides service-specific performance metrics.
For example:
Suppose our application uses EC2, RDS, and an ALB.
We can monitor EC2 CPU, RDS database connections, and ALB request or error metrics.
Together these metrics help us understand application health.
Example:
CloudWatch
├— EC2 → CPU
├— RDS → Connections
└— ALB → Requests / Errors

Simple ga gurthupettuko:
Each AWS service = Its own useful CloudWatch metrics
AWS maintains a list of services that publish CloudWatch metrics, including EC2, EBS, RDS, CloudFront, CloudWatch Logs, Synthetics, and others. **A**  AWS Documentation

---

8. What is Metric Math?
Answer:
Metric Math allows us to perform calculations using one or more CloudWatch metrics.
Instead of looking at separate metrics, we can create a calculated result from them.
It is useful when the required monitoring value is not directly available as a single metric.
For example:
Suppose we have successful requests and total requests.
We can use Metric Math to calculate a success percentage.
This calculated value can then be displayed on a dashboard or used for monitoring.
Example:
Successful Requests
+
Total Requests
↓
Metric Math
↓
Success %
Simple ga gurthupettuko:
Metric Math = Calculate new monitoring values

---

9. What is a CloudWatch Alarm?
Answer:
A CloudWatch Alarm continuously watches a metric or supported monitoring data against a configured condition.
When the condition is met, the alarm changes its state and can perform a configured action.
Alarms are commonly used to notify teams or trigger automated responses.
For example:
Suppose our EC2 CPU normally stays below 70%.
We create an alarm for CPU utilization greater than 80%.
If CPU crosses the threshold, the alarm can enter the ALARM state.
Example:
EC2 CPU
↓
> 80%
↓
CloudWatch Alarm
↓
ALARM
↓
Action / Notification
Simple ga gurthupettuko:
Alarm = Watch metric + Condition + Action

CloudWatch alarms have states such as **OK, ALARM, and INSUFFICIENT_DATA**. **A**  AWS Documentation

---

10. What are the states of a CloudWatch Alarm?
Answer:
The main CloudWatch alarm states are OK, ALARM, and INSUFFICIENT_DATA.
OK means the monitored condition is currently healthy.
ALARM means the configured condition has been breached, while INSUFFICIENT_DATA means there is not enough data to determine the state.
For example:
Suppose we monitor EC2 CPU with an 80% threshold.
CPU below the threshold can keep the alarm in OK, while CPU above the configured condition can move it to ALARM.
If CloudWatch cannot determine the state because data is missing, it can show INSUFFICIENT_DATA.
Example:
CPU Metric
↓

OK
ALARM
INSUFFICIENT

└_____________┘

Simple ga gurthupettuko:
OK = Healthy | ALARM = Problem | INSUFFICIENT = Not enough data

---

11. What is an Alarm Threshold?
Answer:
A threshold is the value that CloudWatch compares against a metric.
When the metric crosses the configured condition, the alarm can change state.
The threshold depends on what we want to monitor.
For example:
Suppose our application server should normally stay below 80% CPU.
We configure 80% as the threshold for the CPU alarm.
If CPU stays above the configured condition for the required evaluation period, the alarm can trigger.
Example:
CPU
↓
80% Threshold
↓
Above threshold
↓
Alarm
Simple ga gurthupettuko:
Threshold = Limit used by alarm

---

12. What are Period and Evaluation Periods in CloudWatch Alarms?
Answer:
The period defines the length of time over which CloudWatch evaluates a metric.
Evaluation periods determine how many periods CloudWatch considers when deciding the alarm state.
These settings help avoid triggering alarms because of a very short temporary spike.
For example:
Suppose CPU suddenly becomes high for a few seconds.
Instead of immediately treating every small spike as a problem, we can configure an appropriate period and evaluation requirement.
This makes the alarm more useful for real application conditions.
Example:
Metric
↓
Period
↓
Evaluation Periods
↓
Alarm Decision
Simple ga gurthupettuko:
Period = Time window | Evaluation = How many periods are checked

---

13. What is the difference between Metric and Alarm?
Answer:
A metric is the actual monitoring data collected by CloudWatch.
An alarm watches that metric and checks whether a configured condition is met.
So a metric provides information, while an alarm reacts to that information.
For example:
EC2 CPUUtilization is a metric.
An alarm can monitor that metric and trigger when CPU remains above a selected threshold.
The metric tells us what is happening, while the alarm tells us when action may be required.
Example:
EC2
↓
Metric
↓
CPU = 85%
↓
Alarm
↓
ALARM
Simple ga gurthupettuko:
Metric = Data | Alarm = Action based on data

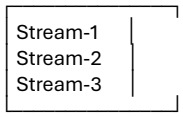
---

14. What are CloudWatch Logs?
Answer:
CloudWatch Logs is used to collect, store, search, and monitor log data.
Applications, EC2 instances, Lambda functions, and other AWS services can send logs to CloudWatch Logs.
It is useful for troubleshooting application and infrastructure problems.

For example:
Suppose our application running on EC2 generates error messages.
We can send those logs to CloudWatch Logs instead of checking every server manually.
Developers or administrators can then search the logs to identify problems.
Example:
Application
↓
CloudWatch Logs
↓
Log Group
↓
Log Stream
↓
Error Logs
Simple ga gurthupettuko:
CloudWatch Logs = Central place for log data

---

15. What is a Log Group?
Answer:
A Log Group is a container that organizes related CloudWatch log streams.
We normally create separate log groups for different applications, services, or environments.
A log group can also have settings such as retention and metric filters.
For example:
Suppose we have a production application.
We can create a log group such as /app/production for its logs.
The individual server or application log streams can then exist inside that log group.
Example:
Log Group
/app/production
↓



Simple ga gurthupettuko:
Log Group = Container for related logs

---

16. What is a Log Stream?
Answer:
A Log Stream is a sequence of log events from a particular source within a CloudWatch Log Group.
It helps separate logs coming from different instances, containers, or application sources.
Multiple log streams can exist inside one log group.
For example:
Suppose three EC2 servers send application logs to one log group.
CloudWatch can maintain separate log streams for those different sources.
This makes it easier to identify where a particular log came from.
Example:
Log Group
↓
├— EC2-1 Stream
├— EC2-2 Stream
└— EC2-3 Stream
Simple ga gurthupettuko:
Log Stream = Sequence of logs from a source

---

17. What is Log Retention?
Answer:
Log retention controls how long CloudWatch keeps logs in a log group.
We can choose an appropriate retention period based on business, troubleshooting, and compliance requirements.
After the configured retention period, old log events are automatically deleted.
For example:
Suppose application logs are needed only for 30 days.
We can configure the log group retention accordingly.
CloudWatch will automatically remove older log data after the retention period.
Example:
Application Logs
↓
CloudWatch
↓
Retention = 30 Days

↓
Old Logs → Deleted
Simple ga gurthupettuko:
Retention = How long logs are kept

---

18. What is CloudWatch Logs Insights?
Answer:
CloudWatch Logs Insights is used to interactively search and analyze log data.
We can write queries to filter logs, find errors, calculate values, and identify patterns.
It is very useful when troubleshooting large amounts of application logs.
For example:
Suppose our application has thousands of logs and we want to find HTTP 500 errors.
Instead of manually opening every log stream, we can query the logs using Logs Insights.
The query can return only the relevant error records.
Example:
Log Groups
↓
Logs Insights
↓
Query
↓
Filter Errors
↓
Results
Simple ga gurthupettuko:
Logs Insights = Search + Query + Analyze logs

---

19. What is a Metric Filter?
Answer:
A Metric Filter searches incoming log events for a specified pattern and can publish a CloudWatch metric based on matching events.
This allows us to convert useful information from logs into measurable metrics.
Those metrics can then be used with CloudWatch alarms.
For example:
Suppose application logs contain the word ERROR whenever an application failure occurs.
We can create a metric filter that counts matching ERROR messages.
Then we can create an alarm when the error count becomes too high.
Example:
Application Logs
↓
Metric Filter
↓
ERROR Count
↓
CloudWatch Metric
↓
Alarm
Simple ga gurthupettuko:
Metric Filter = Convert matching logs into metrics

---

20. What is a Subscription Filter?
Answer:
A subscription filter controls which log events are sent from CloudWatch Logs to a destination.
It can filter log data based on patterns and stream matching logs to supported destinations.
This is useful when another service needs to process logs in near real time.
For example:
Suppose we want specific application logs to be processed automatically.
We create a subscription filter that selects the required log events.
The matching logs can then be sent to a supported destination such as Lambda or Kinesis.
Example:
CloudWatch Logs
↓
Subscription Filter
↓
Matching Logs
↓
Lambda / Kinesis
Simple ga gurthupettuko:
Subscription Filter = Send selected logs somewhere else

---

21. What is CloudWatch Live Tail?

Answer:
Live Tail allows us to view incoming log events in near real time.
It is useful when we are actively troubleshooting an application and want to see new log entries as they arrive.
We can monitor selected log groups without waiting to manually refresh log data.
For example:
Suppose we deploy a new application version and immediately start seeing errors.
We can open Live Tail and watch the application's new log events as they arrive.
This helps us quickly identify the issue.
Example:
Application
↓
New Logs
↓
CloudWatch Live Tail
↓
Real-Time View
Simple ga gurthupettuko:
Live Tail = Watch incoming logs live

---

22. What is CloudWatch Logs Anomaly Detection?
Answer:
Logs anomaly detection helps identify unusual patterns or behavior in log data.
It can help us find log patterns that differ from normal activity.
This is useful for discovering unexpected application behavior without manually checking every log.
For example:
Suppose an application normally generates a small number of specific errors.
Suddenly the error pattern changes significantly.
Anomaly detection can help identify that unusual behavior.
Example:
Normal Logs
↓
CloudWatch
↓
Anomaly Detection
↓
Unusual Pattern
↓
Investigation
Simple ga gurthupettuko:
Anomaly Detection = Find unusual behavior

---

23. What is CloudWatch Application Signals?
Answer:
Application Signals is a CloudWatch capability used to monitor application services and their operational health.
It provides information such as service performance, availability, latency, faults, and dependencies.
It helps teams understand application health from a service-level perspective.
For example:
Suppose our application has frontend, API, and database-related services.
Application Signals can help us understand which service has high latency or errors.
We can then drill into the affected service for troubleshooting.
Example:
Application
↓
Application Signals
↓
Services
↓
Latency / Errors / Availability
Simple ga gurthupettuko:
Application Signals = Monitor application service health
AWS describes Application Signals as providing service views, service details, application topology, SLIs/SLOs, and related telemetry. **A**  AWS Documentation

---

24. What is the Application Map in CloudWatch?
Answer:
Application Map provides a visual view of application services and their dependencies.
It helps us understand how different components communicate with each other.
This makes it easier to identify which service or dependency may be causing an application problem.

For example:
Suppose a frontend calls an API service and the API communicates with another service.
The Application Map can show these relationships so we can understand the application topology.
If one service becomes unhealthy, we can investigate its dependencies.
Example:
User
↓
Frontend
↓
API Service
↓
Database Service
Simple ga gurthupettuko:
Application Map = Visual application topology

The Application Map is the current CloudWatch application-topology view associated with Application Signals. **A** AWS Documentation

---

25. What are SLOs in CloudWatch Application Signals?
Answer:
SLO stands for Service Level Objective.
It defines a target for the reliability or performance of a service, such as availability or latency.
CloudWatch Application Signals can show SLI health against configured SLO objectives.
For example:
Suppose we want our API to be available 99.9% of the time.
We can define an SLO around that target.
CloudWatch can then help us monitor whether the service is meeting the objective.
Example:
API Service
↓
SLO = 99.9% Availability
↓
CloudWatch
↓
SLO Health
Simple ga gurthupettuko:
SLO = Target we want our service to achieve

---

26. What is CloudWatch Synthetics?
Answer:
CloudWatch Synthetics is used to monitor applications by running automated canaries.
Canaries simulate user actions or requests and check whether an application is working correctly.
This helps detect availability and functional problems before real users report them.
For example:
Suppose our website should always load successfully.
A Synthetics canary can periodically open the website and check the expected response.
If the check fails, we can investigate or trigger an alert.
Example:
Canary
↓
Website
↓
Test Request
↓
Success / Failure
↓
CloudWatch
Simple ga gurthupettuko:
Synthetics = Automated user-like testing

---

27. What is a CloudWatch Canary?
Answer:
A Canary is an automated script that runs periodically to test an application or endpoint.
It can perform actions such as opening a webpage or making requests and checking the result.
Canaries help detect application availability and functional problems continuously.
For example:
Suppose our login page is important to users.
We can create a canary that periodically opens the login page and checks whether it works.
If the canary fails, we know there may be an application problem.

Example:
Canary
↓
Login Page
↓
Test
↓
Success / Failure
Simple ga gurthupettuko:
Canary = Automated application checker

---

28. What is CloudWatch Network Monitoring?
Answer:
CloudWatch Network Monitoring provides monitoring capabilities for network health and performance.
It can help us understand network connectivity, latency, and packet-loss-related information depending on the monitoring feature used.
This is useful for troubleshooting network problems.
For example:
Suppose an application is experiencing slow network communication.
We can use network monitoring information to investigate latency or packet loss.
This helps determine whether the issue is related to network connectivity.
Example:
Application
↓
Network
↓
CloudWatch Network Monitoring
↓
Latency / Packet Loss
↓
Troubleshooting
Simple ga gurthupettuko:
Network Monitoring = Monitor network health
CloudWatch Network Synthetic Monitor dashboards can show network health indicators, round-trip time, and packet loss for configured monitors. **A** AWS Documentation

---

29. What is CloudWatch Container Insights?
Answer:
Container Insights provides monitoring information for containerized applications and infrastructure.
It can collect and display useful performance information for supported container environments.
It helps teams monitor container resources and troubleshoot container-related problems.
For example:
Suppose our application runs using containers on ECS or EKS.
We need to know whether containers are using too much CPU or memory.
Container Insights helps provide monitoring visibility for the container environment.
Example:
ECS / EKS
↓
Container Insights
↓
CPU / Memory / Performance
↓
CloudWatch
Simple ga gurthupettuko:
Container Insights = Monitor containers

---

30. What is CloudWatch Lambda Insights?
Answer:
Lambda Insights provides enhanced monitoring information for AWS Lambda functions.
It helps us understand function performance and resource behavior beyond basic Lambda metrics.
This is useful when troubleshooting slow or resource-intensive Lambda functions.
For example:
Suppose a Lambda function is taking longer than expected to complete.
We can use Lambda monitoring information to investigate its performance and resource usage.
This helps identify possible bottlenecks.
Example:
Lambda
↓

Lambda Insights
↓
Performance Data
↓
CloudWatch
Simple ga gurthupettuko:
Lambda Insights = Detailed Lambda monitoring

---

31. What is CloudWatch Application Insights?
Answer:
CloudWatch Application Insights helps monitor applications and their supporting AWS resources.
It can help configure useful metrics, logs, alarms, and dashboards for supported application components.
It is useful when we want application-level monitoring instead of monitoring individual resources separately.
For example:
Suppose an application uses EC2, load balancers, databases, and other components.
Application Insights can help monitor these related components together.
It can also help identify problems by correlating monitoring information.
Example:
Application
├— EC2
├— ELB
├— Database
└— Logs
↓
Application Insights
↓
Monitoring
Simple ga gurthupettuko:
Application Insights = Application-level monitoring
AWS documents Application Insights as automatically setting up relevant metrics, logs, alarms, and dashboards for supported application resources. **A**  AWS Documentation

---

32. What is CloudWatch EventBridge integration?
Answer:
CloudWatch integrates with Amazon EventBridge to react to events occurring in AWS services and applications.
EventBridge can receive events and route them to targets based on rules.
This allows us to automate actions when specific AWS events occur.
For example:
Suppose an EC2 instance changes state to stopped.
An EventBridge rule can detect that event and invoke a target such as Lambda.
The target can then perform an automated action.
Example:
AWS Event
↓
EventBridge Rule
↓
Target
↓
Lambda / SNS / Other Service
Simple ga gurthupettuko:
EventBridge = React to AWS events

---

33. What is the difference between CloudWatch Metrics and CloudWatch Logs?
Answer:
CloudWatch Metrics are numerical measurements used to monitor resource or application performance.
CloudWatch Logs contain detailed textual or structured log events generated by applications and services.
Metrics are useful for graphs and alarms, while logs are useful for detailed troubleshooting.
For example:
EC2 CPU utilization is a metric.
An application error message such as Database connection failed is a log entry.
We can use both together to understand why an application is having problems.
Example:
CloudWatch
├— Metrics → CPU = 90%
└— Logs
↓
Database Error

Simple ga gurthupettuko:
Metrics = Numbers | Logs = Detailed messages

---

34. How does CloudWatch help with EC2 monitoring?
Answer:
CloudWatch collects monitoring metrics from EC2 instances and allows us to visualize and alarm on them.
Common metrics include CPU utilization and network-related metrics, with additional OS-level information available through agents/configuration.
We can create dashboards and alarms to monitor EC2 health.
For example:
Suppose our EC2 CPU continuously reaches 90%.
We can create a CloudWatch alarm for high CPU utilization.
The alarm can notify the operations team or participate in an automated response.
Example:
EC2
↓
CloudWatch
↓
CPU Metric
↓
Alarm
↓
Notification
Simple ga gurthupettuko:
EC2 + CloudWatch = Monitor server health

---

35. How does CloudWatch help with Auto Scaling?
Answer:
CloudWatch metrics and alarms can be used with Auto Scaling to help determine when scaling actions should happen.
For example, an alarm can detect high or low application load.
The Auto Scaling configuration can then respond by adding or removing EC2 instances according to its scaling policy.
For example:
Suppose our application normally runs with two EC2 instances.
If average CPU becomes consistently high, a CloudWatch-based scaling condition can contribute to adding instances.
When demand decreases, scaling can reduce the number of instances.
Example:
EC2
↓
CloudWatch CPU
↓
Alarm / Scaling Condition
↓
Auto Scaling Group
↓
Add / Remove Instances
Simple ga gurthupettuko:
CloudWatch = Monitoring | ASG = Scaling

---

36. How does CloudWatch help troubleshoot an application?
Answer:
CloudWatch gives us metrics, logs, alarms, and application monitoring information to investigate problems.
We can first check metrics to identify abnormal resource behavior and then inspect logs for detailed errors.
This combination helps us find the likely cause of an application issue.
For example:
Suppose users report that our application is slow.
We can check EC2 CPU, ALB latency, database metrics, and application logs.
By comparing these signals, we can identify which component is causing the problem.
Example:
User Reports Slow App
↓
CloudWatch Metrics
↓
CloudWatch Logs
↓
Find Problem
↓
Fix Resource / Application
Simple ga gurthupettuko:
Troubleshooting = Metrics + Logs + Alarms + Insights

---

37. What is CloudWatch Agent?

Answer:
The CloudWatch Agent is software that can collect additional system-level metrics and logs from supported servers.
It can collect information that is not available through basic monitoring alone, depending on the configuration.
It is commonly used when we need deeper OS-level monitoring.
For example:
Suppose an EC2 instance needs memory and disk-space monitoring.
We can install and configure the CloudWatch Agent on the instance to collect those details.
The collected data can then be sent to CloudWatch.
Example:
EC2
↓
CloudWatch Agent
↓
Memory / Disk / Logs
↓
CloudWatch
Simple ga gurthupettuko:
CloudWatch Agent = Extra OS-level metrics + logs

---

38. What is a Custom Metric?
Answer:
A Custom Metric is a metric that we publish ourselves instead of relying only on AWS-provided metrics.
Applications can publish business or application-specific measurements to CloudWatch.
This allows us to monitor values that AWS services do not automatically provide.
For example:
Suppose our shopping application needs to monitor the number of successful orders per minute.
AWS may not provide that exact business metric automatically.
Our application can publish an OrdersProcessed custom metric to CloudWatch.
Example:
Application
↓
Custom Metric
↓
OrdersProcessed
↓
CloudWatch
↓
Dashboard / Alarm
Simple ga gurthupettuko:
Custom Metric = Our own application/business metric

---

39. Can CloudWatch alarms be added to a Dashboard?
Answer:
Yes, CloudWatch alarms can be displayed on CloudWatch dashboards.
This allows us to see important alarm states together with related metrics.
It is useful for creating a single monitoring screen for an application.
For example:
Suppose our dashboard contains EC2 CPU and RDS metrics.
We can also add the related high-CPU and database alarms to the same dashboard.
This gives the operations team a quick view of application health.
Example:
CloudWatch Dashboard
├— EC2 CPU
├— RDS Metric
├— ALB Metric
└— Alarms

CloudWatch supports adding alarms, including their graphs, to dashboards. **A**  AWS Documentation
Simple ga gurthupettuko:
Dashboard = Metrics + Alarms in one place

---

40. What are the most important CloudWatch console areas to remember?
Answer:
The most important areas are Dashboards, Metrics, Alarms, Logs, Logs Insights, Application Signals, Synthetics, and monitoring/insight features.
Metrics help us measure performance, Logs help troubleshoot detailed events, and Alarms help detect conditions that require action.
Application Signals and Synthetics provide deeper application-level monitoring.
For example:

Suppose our production application is running on EC2 behind an ALB with RDS.
We can use Metrics and Dashboards for health, Logs for errors, Alarms for alerts, and Synthetics for availability testing.
Application Signals can provide a service-level view when the application is instrumented.
Example:
CloudWatch
│
├── Dashboard → Overall View
├── Metrics → Performance
├── Alarms → Alerts
├── Logs → Troubleshooting
├── Logs Insights → Log Queries
├── Application Signals → App Health
└── Synthetics → Automated Testing
Simple ga gurthupettuko:
CloudWatch = Metrics + Logs + Alarms + Dashboards + Application Monitoring

---